PHASE 3 DOCUMENTATION REWRITE

Windows 98 Portfolio OS

Apps and Features Reference — Explained From Current Code

Document	Apps and Features Reference — Explained From Current Code
System	Windows 98 Portfolio OS
Generated	June 20, 2026 — from docs/apps and repository source
Source of truth	Repository source files are authoritative; Markdown docs are background reference where details differ.
Page geometry	US Letter, 1 inch margins, repeating table header rows.

> This phase focuses on every user-facing app and feature window. The Markdown app pages are used as source notes, but the current TypeScript files and app registry are treated as the authority for app count, launch behavior, Safe Mode availability, payload shape, and code highlights.

1 What This Phase Covers

The app layer is the part of the portfolio OS the user touches most directly. It includes filesystem tools, editors, creative apps, media players, browser/network utilities, games, portfolio content windows, and built-in Help. The current registry contains 25 app definitions and each has a matching documentation page under docs/apps.

The important implementation pattern is consistent: appDefinitions owns launch metadata, App.tsx lazy-loads every app component, renderAppWindow maps app IDs to components and custom payload handling, and each app decides how much OS surface it needs through useOs or narrowly injected props.

Current fact	Verified from source
25 registered app definitions	src/data/apps.ts
25 individual app Markdown pages	docs/apps/*.md
24 per-app CSS files	src/components/apps/*App.css
9 multi-instance apps: explorer, terminal, notepad, wordpad, imageViewer, mediaPlayer, videoPlayer, dosGame, projectDetails	singleton: false in appDefinitions
6 unavailable in Safe Mode: internetExplorer, mediaPlayer, videoPlayer, soundRecorder, network, dosGame	safeModeAvailable: false in appDefinitions
42 MS-DOS command labels and aliases	src/os/commands.ts executeCommand switch

2 App Contract and Launch Flow

Every app is a React component rendered inside WindowFrame, but the launch contract is data-first. appDefinitions gives each app an id, title, icon, default rectangle, singleton policy, and Safe Mode policy. Launch surfaces then pass appId plus an optional WindowPayload.

Launch source	How it reaches an app window
Desktop icons	desktopIconDefs entries call openApp(appId, payload); filesystem shortcuts under C:\Windows\Desktop are mirrored into desktop icons.
Start menu	startMenuModel stores app IDs and payloads for Programs, Documents, Settings, Portfolio, Run, and shutdown actions.
Run dialog	Resolves command keywords, URLs, filesystem paths, or Terminal fallback commands.
Explorer/openNode	openTargetFor maps folders and file extensions to explorer, notepad, wordpad, imageViewer, mediaPlayer, videoPlayer, internetExplorer, or terminal.
Terminal	executeCommand returns openApp effects for commands like start, notepad, mspaint, kodakimg, vidplay, iexplore, and calc.
App-to-app navigation	Projects opens Project Details, Image Viewer opens Paint, Sound Recorder opens Media Player, IE opens Network from the offline page.

`src/App.tsx` *lazy app registration and renderAppWindow mapping*

```
const AboutApp = lazy(() => import('./components/apps/AboutApp').then((m) => ({ default: m.A
const CalculatorApp = lazy(() => import('./components/apps/CalculatorApp').then((m) => ({ de
const ContactApp = lazy(() => import('./components/apps/ContactApp').then((m) => ({ default:
const ControlPanelApp = lazy(() =>
  import('./components/apps/ControlPanelApp').then((m) => ({ default: m.ControlPanelApp })),
)
const CreditsApp = lazy(() => import('./components/apps/CreditsApp').then((m) => ({ default:
const ExplorerApp = lazy(() => import('./components/apps/ExplorerApp').then((m) => ({ defaul
const GalleryApp = lazy(() => import('./components/apps/GalleryApp').then((m) => ({ default:
const HelpApp = lazy(() => import('./components/apps/HelpApp').then((m) => ({ default: m.Hel
const ImageViewerApp = lazy(() => import('./components/apps/ImageViewerApp').then((m) => ({
const InternetExplorerApp = lazy(() =>
  import('./components/apps/InternetExplorerApp').then((m) => ({ default: m.InternetExplorer
)
const MediaPlayerApp = lazy(() => import('./components/apps/MediaPlayerApp').then((m) => ({
const JsDosGameApp = lazy(() =>
  import('./components/apps/JsDosGameApp').then((m) => ({ default: m.JsDosGameApp })),
)
const MinesweeperApp = lazy(() => import('./components/apps/MinesweeperApp').then((m) => ({
const NetworkApp = lazy(() => import('./components/apps/NetworkApp').then((m) => ({ default:
const NotepadApp = lazy(() => import('./components/apps/NotepadApp').then((m) => ({ default:
const PaintApp = lazy(() => import('./components/apps/PaintApp').then((m) => ({ default: m.P
const ProjectDetailsApp = lazy(() =>
  import('./components/apps/ProjectDetailsApp').then((m) => ({ default: m.ProjectDetailsApp
)
const ProjectsApp = lazy(() => import('./components/apps/ProjectsApp').then((m) => ({ defaul
const RecycleBinApp = lazy(() => import('./components/apps/RecycleBinApp').then((m) => ({ de
const RunDialogApp = lazy(() => import('./components/apps/RunDialogApp').then((m) => ({ defa
const SoundRecorderApp = lazy(() =>
  import('./components/apps/SoundRecorderApp').then((m) => ({ default: m.SoundRecorderApp })
)
const TaskManagerApp = lazy(() => import('./components/apps/TaskManagerApp').then((m) => ({
const TerminalApp = lazy(() => import('./components/apps/TerminalApp').then((m) => ({ defaul
const VideoPlayerApp = lazy(() => import('./components/apps/VideoPlayerApp').then((m) => ({
```

```
const WordPadApp = lazy(() => import('./components/apps/WordPadApp').then((m) => ({ default:

function renderAppWindow(win: WindowState, openApp: (appId: AppId, payload?: WindowPayload)
  const props = { windowId: win.instanceId, payload: win.payload }
  switch (win.appId) {
    case 'explorer':
      return <ExplorerApp {...props} />
    case 'recycleBin':
      return <RecycleBinApp />
    case 'terminal':
      return <TerminalApp {...props} />
    case 'notepad':
      return <NotepadApp {...props} />
    case 'wordpad':
      return <WordPadApp {...props} />
    case 'paint':
      return <PaintApp {...props} />
    case 'imageViewer':
      return <ImageViewerApp key={win.payload?.filePath ?? win.instanceId} {...props} />
    case 'internetExplorer':
      return <InternetExplorerApp {...props} />
    case 'mediaPlayer':
      return <MediaPlayerApp key={win.payload?.filePath ?? win.payload?.url ?? win.instanceI
    case 'videoPlayer':
      return <VideoPlayerApp key={win.payload?.filePath ?? win.payload?.url ?? win.instanceI
    case 'gallery':
      return <GalleryApp />
    case 'soundRecorder':
      return <SoundRecorderApp />
    case 'controlPanel':
      return <ControlPanelApp key={win.payload?.controlPanelSection ?? 'controlPanel'} {...p
    case 'network':
      return <NetworkApp />
    case 'run':
      return <RunDialogApp />
    case 'taskManager':
```

```
      return <TaskManagerApp />
    case 'calculator':
      return <CalculatorApp />
    case 'minesweeper':
      return <MinesweeperApp />
    case 'dosGame':
      return <JsDosGameApp {...props} />
    case 'about':
      return <AboutApp />
    case 'contact':
      return <ContactApp />
    case 'projects':
      return <ProjectsApp openApp={openApp} />
    case 'projectDetails':
      return <ProjectDetailsApp projectId={win.payload?.projectId} />
    case 'credits':
      return <CreditsApp />
    case 'help':
      return <HelpApp />
```

`src/os/filesystem.ts` *file association routing*

```
const TEXT_EXTENSIONS = new Set(['txt', 'ini', 'log', 'inf', 'bat', 'md'])
const DOCUMENT_EXTENSIONS = new Set(['doc', 'docx', 'rtf'])
const IMAGE_EXTENSIONS = new Set(['bmp', 'png', 'jpg', 'jpeg', 'gif'])
const AUDIO_EXTENSIONS = new Set(['wav', 'mp3', 'mid'])
const VIDEO_EXTENSIONS = new Set(['mp4', 'avi', 'webm', 'mov', 'mkv', 'ogg'])
const WEB_EXTENSIONS = new Set(['url', 'htm', 'html'])

export function openTargetFor(node: FsNode): { appId: AppId; payload: WindowPayload } | null
  if (node.appId) {
    return { appId: node.appId, payload: node.appPayload ?? {} }
  }
  if (node.kind === 'folder') {
    return { appId: 'explorer', payload: { path: node.path } }
  }
  const ext = extensionOf(node.name)
  if (TEXT_EXTENSIONS.has(ext)) {
    return { appId: 'notepad', payload: { filePath: node.path } }
  }
  if (DOCUMENT_EXTENSIONS.has(ext)) {
    return { appId: 'wordpad', payload: { filePath: node.path } }
  }
  if (IMAGE_EXTENSIONS.has(ext)) {
    return { appId: 'imageViewer', payload: { filePath: node.path } }
  }
  if (AUDIO_EXTENSIONS.has(ext)) {
    return { appId: 'mediaPlayer', payload: { filePath: node.path } }
  }
  if (VIDEO_EXTENSIONS.has(ext)) {
    return { appId: 'videoPlayer', payload: { filePath: node.path } }
```

3 Complete App Registry

App ID	Window title	Window / policy	Primary component
explorer	My Computer	x:130,y:72,width:650,height:450; singleton false; safe true	src/components/apps/ExplorerApp.tsx
recycleBin	Recycle Bin	x:300,y:100,width:520,height:360; singleton true; safe true	src/components/apps/RecycleBinApp.tsx
terminal	MS-DOS Prompt	x:210,y:112,width:580,height:380; singleton false; safe true	src/components/apps/TerminalApp.tsx
notepad	Notepad	x:220,y:112,width:560,height:400; singleton false; safe true	src/components/apps/NotepadApp.tsx
wordpad	WordPad	x:190,y:86,width:680,height:500; singleton false; safe true	src/components/apps/WordPadApp.tsx
paint	Paint	x:170,y:80,width:680,height:500; singleton true; safe true	src/components/apps/PaintApp.tsx
imageViewer	Imaging Preview	x:230,y:76,width:620,height:480; singleton false; safe true	src/components/apps/ImageViewerApp.tsx
internetExplorer	Internet Explorer	x:190,y:84,width:700,height:480; singleton true; safe false	src/components/apps/InternetExplorerApp.tsx
mediaPlayer	Media Player	x:320,y:110,width:470,height:440; singleton false; safe false	src/components/apps/MediaPlayerApp.tsx
videoPlayer	Video Player	x:240,y:86,width:640,height:470; singleton false; safe false	src/components/apps/VideoPlayerApp.tsx
gallery	My Pictures	x:250,y:100,width:560,height:430; singleton true; safe true	src/components/apps/GalleryApp.tsx

App ID	Window title	Window / policy	Primary component
soundRecorder	Sound Recorder	x:330,y:150,width:440,height:260; singleton true; safe false	src/components/apps/SoundRecorderApp.tsx
controlPanel	Control Panel	x:230,y:92,width:600,height:430; singleton true; safe true	src/components/apps/ControlPanelApp.tsx
network	Network Neighborhood	x:300,y:120,width:520,height:380; singleton true; safe false	src/components/apps/NetworkApp.tsx
run	Run	x:120,y:320,width:400,height:170; singleton true; safe true	src/components/apps/RunDialogApp.tsx
taskManager	Task Manager	x:300,y:130,width:480,height:360; singleton true; safe true	src/components/apps/TaskManagerApp.tsx
calculator	Calculator	x:360,y:150,width:250,height:340; singleton true; safe true	src/components/apps/CalculatorApp.tsx
minesweeper	Minesweeper	x:280,y:96,width:360,height:470; singleton true; safe true	src/components/apps/MinesweeperApp.tsx
dosGame	DOS Game	x:60,y:30,width:860,height:640; singleton false; safe false	src/components/apps/JsDosGameApp.tsx
about	About Me	x:140,y:80,width:460,height:360; singleton true; safe true	src/components/apps/AboutApp.tsx
contact	Contact	x:310,y:150,width:430,height:330; singleton true; safe true	src/components/apps/ContactApp.tsx
projects	My Projects	x:190,y:94,width:580,height:430; singleton true; safe true	src/components/apps/ProjectsApp.tsx
projectDetails	Project Details	x:360,y:116,width:480,height:360; singleton false; safe true	src/components/apps/ProjectDetailsApp.tsx

App ID	Window title	Window / policy	Primary component
credits	Credits	x:280,y:110,width:470,height:320; singleton true; safe true	src/components/apps/CreditsApp.tsx
help	Help	x:200,y:76,width:640,height:470; singleton true; safe true	src/components/apps/HelpApp.tsx

4 Apps by Feature Area

The sections below explain the app surface by feature area. Each entry includes what the user can do, how the current code implements it, how it integrates with the OS store or browser APIs, and the exact source files used as evidence.

4 Shell and filesystem utilities

4.1 My Computer (explorer)

A faithful Win98 file manager over the in-memory virtual filesystem: five view modes, click/Ctrl/Shift selection, cut/copy/paste, rename-in-place, send-to-Desktop, delete-to-Recycle-Bin, and per-window passcode-locked folders.

What users can do: A full file manager over the virtual C: drive: navigation, address entry, tree shortcuts, five view modes, sorting, multi-selection, clipboard operations, inline rename, Recycle Bin delete, desktop shortcuts, and passcode-locked folders.

How it is built: The component keeps UI-only state locally and treats the filesystem as store-owned data. The visible listing is derived with listDirectory plus sortNodes; selection uses anchor-path range logic; context menus are clamped to the window with a root ref.

OS integration: Uses useOs for state.fs, state.clipboard, openNode, setWindowTitle, setClipboard, showMessageBox, and fsOps. All creates, renames, moves, copies, deletes, and shortcut writes pass through fsOps so Explorer and Terminal share the same filesystem rules.

Evidence	Current source
Registry	src/data/apps.ts appDefinitions.explorer; defaultRect x:130,y:72,width:650,height:450; singleton false; safeModeAvailable true
Component	src/components/apps/ExplorerApp.tsx
Styles	src/components/apps/ExplorerApp.css
Markdown source	docs/apps/explorer.md

`src/components/apps/ExplorerApp.tsx` *Explorer selection, sort, and rename helpers*

```
function sortNodes(nodes: FsNode[], key: SortKey): FsNode[] {
  return [...nodes].sort((a, b) ⇒ {
    if (a.kind ≠= b.kind) return a.kind == 'folder' ? -1 : 1
    switch (key) {
      case 'type':
        return a.fileType.localeCompare(b.fileType) || a.name.localeCompare(b.name)
      case 'size':
        return a.size - b.size || a.name.localeCompare(b.name)
      case 'modified':
        return a.modified.localeCompare(b.modified) || a.name.localeCompare(b.name)
      case 'name':
      default:
        return a.name.localeCompare(b.name)
    }
  })
}

export function ExplorerApp({ windowId, payload }: AppProps) {
  const { state, openNode, setWindowTitle, fsOps, setClipboard, showMessageBox } = useOs()
  const [currentPath, setCurrentPath] = useState(() ⇒ normalizePath(payload?.path ?? 'C:\\'
  const [address, setAddress] = useState(currentPath)

  function selectOnClick(path: string, event: { ctrlKey: boolean; metaKey: boolean; shiftKey
    if (event.shiftKey && anchorPath) {
      const from = items.findIndex((node) ⇒ node.path == anchorPath)
      const to = items.findIndex((node) ⇒ node.path == path)
      if (from ≠= -1 && to ≠= -1) {
        const [lo, hi] = from < to ? [from, to] : [to, from]
        setSelectedPaths(items.slice(lo, hi + 1).map((node) ⇒ node.path))
        return
      }
    }
    if (event.ctrlKey || event.metaKey) {
      setSelectedPaths((prev) ⇒ (prev.includes(path) ? prev.filter((p) ⇒ p ≠= path) : [..
```

```
      setAnchorPath(path)
      return
    }
    selectSingle(path)
  }

  function showError(message: string) {
    showMessageBox({ title: 'Windows Explorer', message, icon: 'error', buttons: ['ok'] })
  }

  function commitRename() {
    if (!renamingPath) return
    const node = getNode(state.fs, renamingPath)
    const typed = renameValue.trim()
    setRenamingPath(undefined)
    if (!node || !typed) return
    // Keep the original extension if the user dropped it while renaming a file —
    // otherwise the file loses its app association and won't open ("not a valid
    // Win32 application"). Folders are renamed verbatim.
    const originalExt = node.kind === 'file' ? extensionOf(node.name) : ''
    const next = originalExt && !extensionOf(typed) ? `${typed}.${originalExt}` : typed
    if (next === node.name) return
    const error = fsOps.renameNode(node.path, next)
    if (error) showError(error)
    else selectSingle(joinPath(parentPath(node.path), next))
  }

  function createNew(kind: 'folder' | 'file' | 'document') {
    if (protectedHere) {
      showError('Access is denied. Windows folders are protected.')
      return
    }
```

4.2 Recycle Bin (recycleBin)

The Win98 Recycle Bin over the virtual filesystem: lists everything you've deleted, restores any item to its original location, or permanently empties the bin (which also clears it from browser storage).

What users can do: Lists deleted virtual files, restores individual entries to their original path, or permanently empties the bin after a Win98-style confirmation.

How it is built: It is intentionally simple: no local state, only a render of state.fs.recycle. Each row displays the stored RecycleEntry metadata and calls restoreEntry by id.

OS integration: Uses fsOps.restoreEntry and fsOps.emptyRecycleBin. Explorer and desktop deletes feed entries into the same recycle array, and the desktop icon changes based on whether that array is empty.

Evidence	Current source
Registry	src/data/apps.ts appDefinitions.recycleBin; defaultRect x:300,y:100,width:520,height:360; singleton true; safeModeAvailable true
Component	src/components/apps/RecycleBinApp.tsx
Styles	src/components/apps/RecycleBinApp.css
Markdown source	docs/apps/recycle-bin.md

4.3 Run (run)

The Win98 Start > Run dialog, wired as a universal launcher: type a known command name, a URL, a filesystem path, or any shell command, and it opens the right app - falling back to the MS-DOS prompt for anything else.

What users can do: The Start -> Run dialog, resolving typed keywords, URLs, filesystem paths, and fallback shell commands.

How it is built: A module-level command map handles classic aliases such as cmd, control, mspaint, calc, notepad, wordpad, iexplore, explorer, taskmgr, wmplayer, and resume. The dispatch order is keyword, URL, filesystem path, then Terminal command.

OS integration: Uses state.fs for getNode path checks, openApp for keyword/URL/command payloads, and openNode for file association. It never writes to state itself.

Evidence	Current source
Registry	src/data/apps.ts appDefinitions.run; defaultRect x:120,y:320,width:400,height:170; singleton true; safeModeAvailable true
Component	src/components/apps/RunDialogApp.tsx
Styles	src/components/apps/RunDialogApp.css
Markdown source	docs/apps/run-dialog.md

4.4 Task Manager (taskManager)

The Close Program / processes view - a live list of open windows and simulated system processes, with real End Task / End Process behavior and Canvas-drawn CPU & memory graphs. Killing EXPLORER.EXE really does blue-screen the OS.

What users can do: A Close Program/process manager with Applications, Processes, and Performance tabs, including real End Task behavior and canvas CPU/memory graphs.

How it is built: The process list is derived from a static system process list plus one row per open window. Performance graphs use a rolling sample buffer and Canvas 2D drawing on a timer.

OS integration: Uses state.windows, closeWindow, focusWindow, openApp('run'), showMessageBox, and crashSystem. Ending EXPLORER.EXE triggers a real crashed phase.

Evidence	Current source
Registry	src/data/apps.ts appDefinitions.taskManager; defaultRect x:300,y:130,width:480,height:360; singleton true; safeModeAvailable true
Component	src/components/apps/TaskManagerApp.tsx
Styles	src/components/apps/TaskManagerApp.css
Markdown source	docs/apps/task-manager.md

`src/components/apps/TaskManagerApp.tsx` *Task Manager graph and process actions*

```
    function drawGraph(canvas: HTMLCanvasElement | null, history: number[], color: string) {
      const context = canvas?.getContext('2d')
      if (!canvas || !context) return
      const { width, height } = canvas
      context.fillStyle = '#000'
      context.fillRect(0, 0, width, height)
      context.strokeStyle = '#0b4f0b'
      context.lineWidth = 1
      for (let x = 0; x < width; x += 12) {
        context.beginPath()
        context.moveTo(x + 0.5, 0)
        context.lineTo(x + 0.5, height)
        context.stroke()
      }
      for (let y = 0; y < height; y += 12) {
        context.beginPath()
        context.moveTo(0, y + 0.5)
        context.lineTo(width, y + 0.5)
        context.stroke()
      }
      context.strokeStyle = color
      context.lineWidth = 2
      context.beginPath()
      history.forEach((value, index) => {
        const x = width - (history.length - 1 - index) * 4
        const y = height - (value / 100) * (height - 4) - 2
        if (index === 0) context.moveTo(x, y)
        else context.lineTo(x, y)
      })
      context.stroke()

  function endTask() {
    if (!selectedTask) return
    closeWindow(selectedTask)
```

```
    setSelectedTask(undefined)
  }

  function endProcess() {
    const process = processes.find((row) => row.id === selectedProcess)
    if (!process) return
    if (process.kind === 'app' && process.windowId) {
      closeWindow(process.windowId)
      setSelectedProcess(undefined)
      return
    }
    if (process.kind === 'shell') {
      showMessageBox({
        title: 'Task Manager',
        message: 'Ending EXPLORER.EXE will shut down the Windows shell. Continue?',
        icon: 'warning',
        buttons: ['yes', 'no'],
        onResult: (button) => {
          if (button === 'yes') {
            crashSystem({
              title: 'Windows protection error',
              message: 'The Windows shell (EXPLORER.EXE) was terminated by Task Manager.',
              detail: 'The desktop, taskbar, and all running programs can no longer continue
              stopCode: '0E : 0157 : BFF9DB61',
              crashedAt: new Date().toLocaleTimeString(),
            })
          }
        },
      })
      return
    }
    showMessageBox({
      title: 'Task Manager',
```

5 Editors and creative tools

5.1 Notepad (notepad)

The plain-text editor of the simulated desktop: open a .txt, type, Save, and the bytes land back in the virtual C: drive.

What users can do: A plain-text editor for .txt, .ini, .log, .inf, .bat, and .md files with Save, Save As, word wrap, dirty-title indicator, byte count, and line/column status.

How it is built: The editor is a controlled textarea. useMemo seeds the text from payload.filePath or a portfolio welcome note; every edit flips the saved flag and the window title effect mirrors the dirty state.

OS integration: Uses state.fs to read opened nodes, fsOps.writeFile to persist content, setWindowTitle for the title bar, and showMessageBox for write errors. Save writes the current file or defaults new notes into C:\My Documents.

Evidence	Current source
Registry	src/data/apps.ts appDefinitions.notepad; defaultRect x:220,y:112,width:560,height:400; singleton false; safeModeAvailable true
Component	src/components/apps/NotepadApp.tsx
Styles	src/components/apps/NotepadApp.css
Markdown source	docs/apps/notepad.md

5.2 WordPad (wordpad)

The rich-text editor of the desktop: a multi-page contentEditable document with a live font/size/colour toolbar, bullets and alignment, page breaks - and every bit of formatting is round-tripped to HTML and saved into the virtual C: drive.

What users can do: A multi-page rich-text editor with contentEditable pages, formatting toolbar, fonts, sizes, color palette, alignment, bullets, date/time insert, page breaks, and HTML-backed save/load.

How it is built: WordPad separates pageHtmls state from live editable DOM refs. It preserves Range selection across toolbar clicks, uses execCommand for native editing operations, and uses wordpadFormatting.ts to serialize pages and apply stable inline spans.

OS integration: Reads payload.filePath, loads content through wordPadContentToPages, writes HTML through fsOps.writeFile, updates its window title with a dirty marker, opens Explorer for File Open, and closes through closeWindow.

Evidence	Current source
Registry	src/data/apps.ts appDefinitions.wordpad; defaultRect x:190,y:86,width:680,height:500; singleton false; safeModeAvailable true
Component	src/components/apps/WordPadApp.tsx
Styles	src/components/apps/WordPadApp.css
Markdown source	docs/apps/wordpad.md

`src/components/apps/WordPadApp.tsx` *WordPad inline styling and save path*

```
function applyInlineStyle(style: WordPadInlineStyle) {
  const el = focusEditor()
  if (!el) return
  const range = currentRange()
  if (!range) return
  const nextRange = applyWordPadInlineStyle(el, range, style)
  if (!nextRange) return

  const sel = window.getSelection()
  if (!sel) return
  sel.removeAllRanges()
  sel.addRange(nextRange)
  savedRangeRef.current = nextRange.cloneRange()

  // Taglish note: dito sinisigurado na real inline styles ang nasa document,
  // hindi browser-dependent execCommand output lang.
  syncActive()
  markDirty()
  recomputeStats()
}

function applyFont(family: string) {
  setFontName(family)
  runNativeEditCommand('fontName', family)
  applyInlineStyle({ fontFamily: family })
}

function applyFontSize(pt: string) {
  setFontSize(pt)
  const legacySize = Math.max(1, Math.min(7, Math.round(Number(pt) / 5)))
  runNativeEditCommand('fontSize', String(legacySize))
  applyInlineStyle({ fontSize: `${pt}pt` })
}
```

```
function applyColor(value: string) {
  setColorValue(value)
  setColorOpen(false)
  runNativeEditCommand('foreColor', value)
  applyInlineStyle({ color: value })
}



function save(path = documentPath) {
  const target = path ?? joinPath('C:\\My Documents', normalizeDocumentName(saveAsName))
  const pages = readPagesFromDom()
  const error = fsOps.writeFile(target, { content: wordPadPagesToDocumentHtml(pages) })
  if (error) {
    showError(error)
    return
  }
  setPageHtmls(pages)
  setDocumentPath(target)
  setSaveAsName(baseName(target))
  setDirty(false)
  setStatus(`Saved ${target}`)
}

function saveAs() {
  const folder = documentPath ? parentPath(documentPath) : 'C:\\My Documents'
  save(joinPath(folder, normalizeDocumentName(saveAsName)))
}

function newDocument() {
```

5.3 Paint (paint)

The most technically intricate app in the OS: a full Canvas-based drawing program with nine tools, flood fill, and - its party trick - a hand-written 24-bit BMP encoder so it can save in Paint's authentic native format.

What users can do: A canvas drawing program with pencil, brush, eraser, fill, picker, text, line, rectangle, ellipse, palette, brush sizes, fill toggle, undo/redo, and BMP/PNG/JPG saving.

How it is built: The drawing hot path is imperative: canvas, image snapshots, start point, and history live in refs so pointer movement does not force React re-renders. State is reserved for toolbar choices.

OS integration: Uses payload.filePath to load an image, fsOps.writeFile to save to C:\My Documents\Paint, setWindowTitle for dirty/opened file naming, and showMessageBox for save errors. Image Viewer can hand off to Paint for editing.

Evidence	Current source
Registry	src/data/apps.ts appDefinitions.paint; defaultRect x:170,y:80,width:680,height:500; singleton true; safeModeAvailable true
Component	src/components/apps/PaintApp.tsx
Styles	src/components/apps/PaintApp.css
Markdown source	docs/apps/paint.md

`src/components/apps/PaintApp.tsx` *Paint BMP encoder and flood fill*

```
function canvasToBmpDataUrl(canvas: HTMLCanvasElement): string {
  const ctx = canvas.getContext('2d')
  if (!ctx) return canvas.toDataURL('image/png')
  const { width, height } = canvas
  const { data } = ctx.getImageData(0, 0, width, height)
  const rowSize = Math.floor((24 * width + 31) / 32) * 4
  const pixelArraySize = rowSize * height
  const fileSize = 54 + pixelArraySize
  const buffer = new ArrayBuffer(fileSize)
  const view = new DataView(buffer)
  view.setUint8(0, 0x42) // 'B'
  view.setUint8(1, 0x4d) // 'M'
  view.setUint32(2, fileSize, true)
  view.setUint32(10, 54, true) // pixel data offset
  view.setUint32(14, 40, true) // DIB header size
  view.setInt32(18, width, true)
  view.setInt32(22, height, true) // positive height = bottom-up rows
  view.setUint16(26, 1, true) // planes
  view.setUint16(28, 24, true) // bits per pixel
  view.setUint32(34, pixelArraySize, true)
  view.setInt32(38, 2835, true) // ~72 DPI
  view.setInt32(42, 2835, true)
  let offset = 54
  for (let y = height - 1; y >= 0; y -= 1) {
    for (let x = 0; x < width; x += 1) {
      const i = (y * width + x) * 4
      view.setUint8(offset, data[i + 2]) // B
      view.setUint8(offset + 1, data[i + 1]) // G
      view.setUint8(offset + 2, data[i]) // R
      offset += 3
    }
    for (let pad = width * 3; pad < rowSize; pad += 1) {
      view.setUint8(offset, 0)
      offset += 1
```

```
    }
  }
  const bytes = new Uint8Array(buffer)
  let binary = ''
  const chunk = 8192
  for (let i = 0; i < bytes.length; i += chunk) {
    binary += String.fromCharCode(...bytes.subarray(i, i + chunk))
  }
  return `${btoa(binary)}`
}

function hexToRgb(hex: string): [number, number, number] {
  const clean = hex.replace('#', '')
  const full = clean.length === 3 ? clean.split('').map((c) => c + c).join('') : clean
  const n = parseInt(full, 16)
  return [(n >> 16) & 255, (n >> 8) & 255, n & 255]
}

function toHex(r: number, g: number, b: number): string {
  return `#${[r, g, b].map((v) => v.toString(16).padStart(2, '0')).join('')}`

function floodFill(ctx: CanvasRenderingContext2D, width: number, height: number, sx: number,
  const xi = Math.max(0, Math.min(width - 1, Math.floor(sx)))
  const yi = Math.max(0, Math.min(height - 1, Math.floor(sy)))
  const img = ctx.getImageData(0, 0, width, height)
  const d = img.data
  const start = (yi * width + xi) * 4
  const tr = d[start], tg = d[start + 1], tb = d[start + 2], ta = d[start + 3]
  const [fr, fg, fb] = hexToRgb(hex)
  if (Math.abs(fr - tr) <= 2 && Math.abs(fg - tg) <= 2 && Math.abs(fb - tb) <= 2 && ta === 2
  // Tight tolerance so the fill stops at a drawn shape's outline instead of
  // bleeding across it — clicking inside a circle fills only that circle.
  const tol = 20
  const seen = new Uint8Array(width * height)
  const stack = [yi * width + xi]
  while (stack.length) {
```

```
    const p = stack.pop() as number
    if (seen[p]) continue
    seen[p] = 1
    const i = p * 4
    if (
      Math.abs(d[i] - tr) > tol ||
      Math.abs(d[i + 1] - tg) > tol ||
      Math.abs(d[i + 2] - tb) > tol ||
      Math.abs(d[i + 3] - ta) > tol
    ) {
      continue
    }
    d[i] = fr; d[i + 1] = fg; d[i + 2] = fb; d[i + 3] = 255
    const x = p % width
    const y = (p - x) / width
    if (x + 1 < width) stack.push(p + 1)
    if (x - 1 ⩾ 0) stack.push(p - 1)
    if (y + 1 < height) stack.push(p + width)
    if (y - 1 ⩾ 0) stack.push(p - width)
  }
  ctx.putImageData(img, 0, 0)
}

export function PaintApp({ windowId, payload }: AppProps) {
  const { state, fsOps, setWindowTitle, showMessageBox } = useOs()
```

5.4 Sound Recorder (soundRecorder)

Captures real audio from your microphone, saves it as a .wav into the virtual drive, and hands it straight to Media Player - with a graceful synth-tone fallback when no mic is available.

What users can do: Records microphone audio, saves each finished clip as Recording N.wav, plays the last clip, and opens saved recordings in Media Player.

How it is built: The app is a small state machine: stopped, recording, playing. MediaRecorder, recorded chunks, playback Audio, and counters live in refs; finalizeRecording waits for onstop so the last chunk is not lost.

OS integration: Uses getUserMedia and MediaRecorder when available, falls back to a generated ding WAV, writes clips through fsOps.createFile into C:\My Documents\My Recordings, calls enableAudio, and opens mediaPlayer with the saved file path.

Evidence	Current source
Registry	src/data/apps.ts appDefinitions.soundRecorder; defaultRect x:330,y:150,width:440,height:260; singleton true; safeModeAvailable false
Component	src/components/apps/SoundRecorderApp.tsx
Styles	src/components/apps/SoundRecorderApp.css
Markdown source	docs/apps/sound-recorder.md

`src/components/apps/SoundRecorderApp.tsx` *Sound Recorder finalization and VFS save*

```
async function finalizeRecording() {
  let dataUrl: string | undefined
  if (chunksRef.current.length) {
    const blob = new Blob(chunksRef.current, { type: chunksRef.current[0].type || 'audio/w
    dataUrl = await blobToDataUrl(blob).catch(() => undefined)
  } else {
    dataUrl = await renderSoundToWavDataUrl('ding').catch(() => undefined)
  }
  chunksRef.current = []
  setMode('stopped')
  if (!dataUrl) {
    setStatus('Nothing was captured.')
    return
  }
  lastClipRef.current = dataUrl
  setHasClip(true)
  const index = clipCountRef.current + 1
  clipCountRef.current = index
  setClips(index)
  const name = `Recording ${index}.wav`
  const error = fsOps.createFile(RECORDINGS_DIR, name, { dataUrl, content: `Recorded ${now
  if (error) {
    showMessageBox({ title: 'Sound Recorder', message: error, icon: 'error', buttons: ['ok
    setStatus('Could not save the recording.')
    return
  }
  setStatus(`Saved ${joinPath(RECORDINGS_DIR, name)} — opening Media Player...`)
  openApp('mediaPlayer', { filePath: joinPath(RECORDINGS_DIR, name) })
}

async function record() {
  if (mode === 'recording') return
  enableAudio()
  stopPlayback()
```

```
    chunksRef.current = []
    setSeconds(0)
    try {
      const stream = await navigator.mediaDevices?.getUserMedia({ audio: true })
      if (!stream || typeof MediaRecorder === 'undefined') {
        throw new Error('Recording is unavailable.')
```

6 Media, web, and emulation

6.1 Imaging Preview (imageViewer)

The OS's picture previewer - open any image from the virtual drive, fit it to the window or view it actual-size, step through the rest of the folder, and hand off to Paint for editing.

What users can do: Previews image files from the virtual drive, supports fit/actual-size viewing, browses previous and next images in the folder, copies the path, and opens the current image in Paint.

How it is built: The current path is local state seeded from payload.filePath. The component derives the folder image list with listDirectory and wraps Prev/Next around that ordered list.

OS integration: Read-only on state.fs. Uses openApp('paint', { filePath }) for editing, setWindowTitle for the title bar, and is keyed by payload.filePath so separate images open as clean instances.

Evidence	Current source
Registry	src/data/apps.ts appDefinitions.imageViewer; defaultRect x:230,y:76,width:620,height:480; singleton false; safeModeAvailable true
Component	src/components/apps/ImageViewerApp.tsx
Styles	src/components/apps/ImageViewerApp.css
Markdown source	docs/apps/image-viewer.md

6.2 My Pictures (gallery)

A thumbnail gallery over the My Pictures and My Videos folders - a friendly visual front-end to the virtual drive's media, double-click to open.

What users can do: A thumbnail gallery over My Pictures and My Videos, giving a visual front-end to media files in the virtual drive.

How it is built: It has no local copy of media data. It derives picture and video entries from state.fs with extension filters and renders thumbnails directly from file dataUrls.

OS integration: Uses openNode so file association decides whether a thumbnail opens Image Viewer or Video Player. It is read-only and safe-mode available.

Evidence	Current source
Registry	src/data/apps.ts appDefinitions.gallery; defaultRect x:250,y:100,width:560,height:430; singleton true; safeModeAvailable true
Component	src/components/apps/GalleryApp.tsx
Styles	src/components/apps/GalleryApp.css
Markdown source	docs/apps/gallery.md

6.3 Internet Explorer (internetExplorer)

The desktop's window to a frozen 2010s web - type a URL and it loads the real period-accurate page out of the Wayback Machine, inside an IE5-styled frame.

What users can do: An IE5-style browser that normalizes typed input, embeds period pages through the Wayback Machine, renders a built-in search-results page from Wikipedia search, and shows an offline page when the simulated network is disconnected.

How it is built: History is a manual string stack with an index. resolvePage classifies the current URL as archive, search, or not-found. A faux loading timer drives the throbber and progress bar while fetch/AbortController handle search results safely.

OS integration: Uses state.network.connected, networkOps.recordTraffic, openApp('network') from the offline page, setWindowTitle from the current host, and showMessageBox for printer errors. It is unavailable in Safe Mode.

Evidence	Current source
Registry	src/data/apps.ts appDefinitions.internetExplorer; defaultRect x:190,y:84,width:700,height:480; singleton true; safeModeAvailable false
Component	src/components/apps/InternetExplorerApp.tsx
Styles	src/components/apps/InternetExplorerApp.css
Markdown source	docs/apps/internet-explorer.md

`src/components/apps/InternetExplorerApp.tsx` *Internet Explorer URL normalization and archive mapping*

```
function normalizeUrl(raw: string): string {
  const trimmed = raw.trim()
  if (!trimmed) return HOME
  if (looksLikeSearchQuery(trimmed)) return `http://google.com/search?q=${encodeURIComponent
  if (trimmed.startsWith('about:')) return HOME
  if (/^[a-z]+:/i.test(trimmed) && !/^[a-z]+:\/\//i.test(trimmed)) return trimmed
  if (/^[a-z]+:\/\//i.test(trimmed)) return trimmed
  return `http://${trimmed.replace(/^www\./i, '')}`
}

function isHttpUrl(url: string): boolean {
  try {
    const parsed = new URL(url)
    return parsed.protocol === 'http:' || parsed.protocol === 'https:'
  } catch {
    return false
  }
}

function hostOf(url: string): string {
  try {
    return new URL(url).hostname.replace(/^www\./, '').toLowerCase()
  } catch {
    return ''
  }
}

function archiveUrlFor(url: string): string {
  const host = hostOf(url)
  if (host === 'web.archive.org') return url
  const snapshot = SNAPSHOT_BY_HOST[host] ?? DEFAULT_SNAPSHOT
  return `https://web.archive.org/web/${snapshot}if_/${url}`
}
```

```
function pathOf(url: string): string {
  try {
    return new URL(url).pathname
  } catch {
    return '/'
  }
}
```

6.4 Media Player (mediaPlayer)

Plays the user's audio and video from the virtual drive in one playlist - and can even render the OS's own event sounds on demand via the synth engine.

What users can do: A playlist player for audio and video using one media element, with transport controls, seeking, volume, bundled media, VFS media, and generated system sounds.

How it is built: The playlist is derived from src/data/media.ts plus media files discovered under user folders. Tracks may be real URLs, VFS dataUrls, or synth sound ids that are lazily rendered and cached as WAV data URLs.

OS integration: Uses state.fs and state.audio, setWindowTitle, enableAudio, and showMessageBox. It is read-only on the filesystem and unavailable in Safe Mode.

Evidence	Current source
Registry	src/data/apps.ts appDefinitions.mediaPlayer; defaultRect x:320,y:110,width:470,height:440; singleton false; safeModeAvailable false
Component	src/components/apps/MediaPlayerApp.tsx
Styles	src/components/apps/MediaPlayerApp.css
Markdown source	docs/apps/media-player.md

6.5 Video Player (videoPlayer)

A dedicated player for the C:\My Videos folder - HTML5 video with a thumbnail playlist, transport controls, and graceful "can't play this format" handling.

What users can do: A dedicated video player for C:\My Videos and bundled videos, with a thumbnail playlist, play/pause, stop, seek, volume, and unsupported-format feedback.

How it is built: It seeds an initial track synchronously from payload.filePath when present, derives the playlist from local media and the VFS video folder, then drives UI state from HTML video events.

OS integration: Read-only on state.fs. Uses setWindowTitle, enableAudio, showMessageBox, and file-association routing for video extensions. It is unavailable in Safe Mode.

Evidence	Current source
Registry	src/data/apps.ts appDefinitions.videoPlayer; defaultRect x:240,y:86,width:640,height:470; singleton false; safeModeAvailable false
Component	src/components/apps/VideoPlayerApp.tsx
Styles	src/components/apps/VideoPlayerApp.css
Markdown source	docs/apps/video-player.md

6.6 DOS Game (dosGame)

The window that runs actual DOS games - Wolfenstein 3D and DOOM - by booting a WebAssembly DOSBox emulator inside the desktop. A thin React wrapper around the self-hosted js-dos library.

What users can do: Runs Wolfenstein 3D and DOOM through a self-hosted js-dos WebAssembly DOSBox runtime inside a Win98 window.

How it is built: A module-level state/promise guard injects js-dos script and CSS once. Each app instance mounts window.Dos into a container with kiosk mode and stops the emulator on unmount.

OS integration: Reads payload.url for the .jsdos bundle and payload.windowTitle for window naming. It is multi-instance and unavailable in Safe Mode because it depends on emulator assets.

Evidence	Current source
Registry	src/data/apps.ts appDefinitions.dosGame; defaultRect x:60,y:30,width:860,height:640; singleton false; safeModeAvailable false
Component	src/components/apps/JsDosGameApp.tsx
Styles	src/components/apps/JsDosGameApp.css
Markdown source	docs/apps/jsdos-game.md

`src/components/apps/JsDosGameApp.tsx` *js-dos load-once integration*

```
function loadJsDos(): Promise<void> {
  if (jsDosState === 'ready') return Promise.resolve()
  if (jsDosState === 'loading') return jsDosPromise!

  jsDosState = 'loading'
  jsDosPromise = new Promise<void>((resolve, reject) => {
    if (!document.querySelector('link[href="/js-dos/js-dos.css"]')) {
      const link = document.createElement('link')
      link.rel = 'stylesheet'
      link.href = '/js-dos/js-dos.css'
      document.head.appendChild(link)
    }
    const script = document.createElement('script')
    script.src = '/js-dos/js-dos.js'
    script.onload = () => {
      jsDosState = 'ready'
      resolve()
    }
    script.onerror = () => reject(new Error('Failed to load js-dos'))
    document.head.appendChild(script)
  })

  return jsDosPromise
}

export function JsDosGameApp({ payload }: AppProps) {
  const containerRef = useRef<HTMLDivElement>(null)
  const bundleUrl = payload?.url ?? ''
  const stopRef = useRef<(() => Promise<void>) | null>(null)

  useEffect(() => {
    if (!containerRef.current || !bundleUrl) return
    let cancelled = false
```

```
    loadJsDos()
      .then(() => {
        if (cancelled || !containerRef.current || !window.Dos) return
        const ci = window.Dos(containerRef.current, {
          url: bundleUrl,
          pathPrefix: '/js-dos/emulators/',
          autoStart: true,
          workerThread: false,
          // Hide the js-dos sidebar (save / keyboard / fullscreen / settings)
          // so the game fills the window like a real DOS app.
          kiosk: true,
        })
        stopRef.current = ci.stop.bind(ci)
      })
      .catch((err: unknown) => {
        console.error('[JsDosGameApp] failed to start:', err)
```

7 System configuration and command tools

7.1 Control Panel (controlPanel)

The system settings hub - pick an icon (Display, Mouse, Sounds, Network, Add/Remove Programs, ...) and tweak the live OS: themes, wallpaper, cursor scheme, and audio all change the running desktop immediately.

What users can do: The system settings hub: Display, Mouse, Sounds, System, Network, Date/Time, Keyboard, Printers, and Add/Remove Programs.

How it is built: The selected section is local state seeded from payload.controlPanelSection. A data-driven controlPanelSections list renders applet icons, while SectionRows projects live OS state into property grids.

OS integration: Mutates theme, wallpaper, cursor scheme, audio enabled/muted/volume, and plays soundCatalog items through useOs. It can open Network and Task Manager, and Add/Remove uses message-box confirmation.

Evidence	Current source
Registry	src/data/apps.ts appDefinitions.controlPanel; defaultRect x:230,y:92,width:600,height:430; singleton true; safeModeAvailable true
Component	src/components/apps/ControlPanelApp.tsx
Styles	src/components/apps/ControlPanelApp.css
Markdown source	docs/apps/control-panel.md

7.2 Network Neighborhood (network)

The TCP/IP control panel for the simulated Ethernet adapter - connect, renew a DHCP lease, or pin a static IP, with an authentic "negotiating link" animation.

What users can do: A TCP/IP control panel for the simulated adapter, showing DHCP/static configuration, connection status, packet counters, and a staged connect animation.

How it is built: manualIp and connectStage are local state. beginConnect schedules staged status lines with timers, and applyStaticIp validates dotted-quad input before mutating network state.

OS integration: Uses networkOps.connect, disconnect, renewDhcp, and applyConfig plus showMessageBox for invalid static IPs. It is opened from taskbar network, Start menu, desktop icon, and IE's offline page; unavailable in Safe Mode.

Evidence	Current source
Registry	src/data/apps.ts appDefinitions.network; defaultRect x:300,y:120,width:520,height:380; singleton true; safeModeAvailable false
Component	src/components/apps/NetworkApp.tsx
Styles	src/components/apps/NetworkApp.css
Markdown source	docs/apps/network.md

`src/components/apps/NetworkApp.tsx` *Network staged connect and static IP validation*

```
function beginConnect() {
  if (connectStage !== null) return
  setConnectStage(0)
  CONNECT_STAGES.forEach((_, stage) => {
    timersRef.current.push(
      window.setTimeout(() => setConnectStage(stage), stage * 550),
    )
  })
  timersRef.current.push(
    window.setTimeout(() => {
      networkOps.connect()
      setConnectStage(null)
    }, CONNECT_STAGES.length * 550 + 250),
  )
}

function applyStaticIp() {
  if (!/^\d{1,3}\.\d{1,3}\.\d{1,3}\.\d{1,3}$/.test(manualIp.trim())) {
    showMessageBox({
      title: 'Network',
      message: `'${manualIp}' is not a valid IP address. Use dotted-quad notation, e.g. 19
      icon: 'error',
      buttons: ['ok'],
    })
    return
  }
  networkOps.applyConfig({ connected: true, dhcp: false, ipAddress: manualIp.trim() })
}

const connecting = connectStage !== null
```

7.3 MS-DOS Prompt (terminal)

A genuinely functional DOS command line: dir, cd, tree, type, copy, del, ping, ipconfig, sfc /scannow, start, win, and more - all operating on the real virtual filesystem, with command history and Tab completion. It's also the full-screen shell in "Command Prompt Only" boot mode.

What users can do: A functional MS-DOS Prompt with filesystem commands, networking commands, recovery tools, app launch aliases, command history, Tab completion, streamed output, and Command Prompt Only boot mode.

How it is built: The React component handles keyboard I/O and buffering, while src/os/commands.ts is the pure command engine. executeCommand returns text, cwd changes, stream chunks, and effect descriptors instead of mutating React state directly.

OS integration: Applies command effects through useOs: openApp, fsOps.replaceFs, networkOps, restart, closeWindow, and crashSystem. In dosOnly phase, exit returns to the boot menu rather than closing a window.

Evidence	Current source
Registry	src/data/apps.ts appDefinitions.terminal; defaultRect x:210,y:112,width:580,height:380; singleton false; safeModeAvailable true
Component	src/components/apps/TerminalApp.tsx
Styles	src/components/apps/TerminalApp.css
Markdown source	docs/apps/terminal.md

`src/components/apps/TerminalApp.tsx` *Terminal command effect boundary*

```
function applyEffects(effects: CommandEffect[] | undefined) {
  effects?.forEach((effect) => {
    switch (effect.type) {
      case 'openApp':
        openApp(effect.appId, effect.payload)
        break
      case 'setFs':
        fsOps.replaceFs(effect.fs)
        break
      case 'crash':
        crashSystem(crashFor(effect.criticalPath))
        break
      case 'restart':
        restart(effect.target, { bootProfile: 'warm' })
        break
      case 'networkPing':
        networkOps.recordTraffic(effect.sent, effect.received)
        break
      case 'setNetwork':
        networkOps.applyConfig(effect.network)
        break
      case 'exitWindow':
        if (dosOnly) {
          restart('bootMenu')
        } else {
          closeWindow(windowId)
        }
        break
    }

function runCommand(command: string) {
  const shownPrompt = `${prompt}${command}`
  const output = executeCommand(command, {
    cwd,
```

```
      fs: state.fs,
      network: state.network,
      bootMode: state.bootMode,
      dosOnly,
    })

    if (output.clear) {
      setLines([])
    } else {
      append([shownPrompt, ...output.lines])
    }

    if (output.newCwd) {
      setCwd(output.newCwd)
    }
    applyEffects(output.effects)

    let totalDelay = 0
    output.stream?.forEach((chunk) => {
      totalDelay += chunk.delayMs
      const timer = window.setTimeout(() => {
        append(chunk.lines)
        applyEffects(chunk.effects)
      }, totalDelay)
      timersRef.current.push(timer)
    })
  }

  function submit() {
    runCommand(input)
    if (input.trim()) {
```

8 Games and local-state apps

8.1 Calculator (calculator)

The smallest "real" app in the OS, and the reference example for how an app should read: ~70 lines, one pure function for the math, two pieces of state for the UI. If you want to learn the app pattern, start here.

What users can do: A compact arithmetic calculator with operators, decimals, parentheses, C/CE, and chained results.

How it is built: The keypad is a data array, calculate() is a pure sanitized arithmetic evaluator, and the component keeps only display and expression state.

OS integration: No useOs, no filesystem, no payload. It is the smallest self-contained app and the reference pattern for local-state-only components.

Evidence	Current source
Registry	src/data/apps.ts appDefinitions.calculator; defaultRect x:360,y:150,width:250,height:340; singleton true; safeModeAvailable true
Component	src/components/apps/CalculatorApp.tsx
Styles	src/components/apps/CalculatorApp.css
Markdown source	docs/apps/calculator.md

8.2 Minesweeper (minesweeper)

The Windows classic, fully playable: first-click-safe mine placement, recursive empty-cell reveal, flagging, and three difficulty levels - all in one self-contained component with zero OS dependencies.

What users can do: A full Minesweeper game with first-click safety, difficulty levels, timer, flagging, mine counter, win detection, and loss reveal.

How it is built: The board is Cell[][] state. Mines are placed after the first click so the clicked cell and neighbors are excluded; empty regions reveal through an iterative stack flood algorithm.

OS integration: No useOs and no payload. It is a pure game inside a window frame, launched from the Games menu.

Evidence	Current source
Registry	src/data/apps.ts appDefinitions.minesweeper; defaultRect x:280,y:96,width:360,height:470; singleton true; safeModeAvailable true
Component	src/components/apps/MinesweeperApp.tsx
Styles	src/components/apps/MinesweeperApp.css
Markdown source	docs/apps/minesweeper.md

`src/components/apps/MinesweeperApp.tsx` *Minesweeper mine placement and flood reveal*

```
function placeMines(board: Cell[][], mines: number, safeR: number, safeC: number): Cell[][]
  const rows = board.length
  const cols = board[0].length
  const next = board.map((row) => row.map((cell) => ({ ...cell, mine: false, adjacent: 0 })))
  const forbidden = new Set<string>([`${safeR},${safeC}`, ...neighbors(safeR, safeC, rows, c
  let placed = 0
  let guard = 0
  while (placed < mines && guard < 100000) {
    guard += 1
    const r = Math.floor(Math.random() * rows)
    const c = Math.floor(Math.random() * cols)
    if (forbidden.has(`${r},${c}`) || next[r][c].mine) continue
    next[r][c].mine = true
    placed += 1
  }
  for (let r = 0; r < rows; r += 1) {
    for (let c = 0; c < cols; c += 1) {
      if (next[r][c].mine) continue
      next[r][c].adjacent = neighbors(r, c, rows, cols).filter(([nr, nc]) => next[nr][nc].mi
    }
  }
  return next
}

function floodReveal(board: Cell[][], startR: number, startC: number): Cell[][] {
  const rows = board.length
  const cols = board[0].length
  const next = board.map((row) => row.map((cell) => ({ ...cell })))
  const stack: Array<[number, number]> = [[startR, startC]]
  while (stack.length) {
    const [r, c] = stack.pop() as [number, number]
    const cell = next[r][c]
    if (cell.revealed || cell.flagged) continue
    cell.revealed = true
```

```
    if (cell.adjacent === 0 && !cell.mine) {
      for (const [nr, nc] of neighbors(r, c, rows, cols)) {
        if (!next[nr][nc].revealed) stack.push([nr, nc])
      }
    }
  }
  return next
}

export function MinesweeperApp() {
  const [difficulty, setDifficulty] = useState<Difficulty>('beginner')
  const level = LEVELS[difficulty]
  const [board, setBoard] = useState<Cell[][]>(() => makeBoard(level.rows, level.cols))
  const [gameState, setGameState] = useState<GameState>('ready')
  const [time, setTime] = useState(0)
  const timerRef = useRef<number | null>(null)
```

9 Portfolio content apps

9.1 About Me (about)

The portfolio's "read me first" - a one-screen intro (name, role, headline, highlights) rendered straight from the portfolio data file.

What users can do: The portfolio identity screen: name, role, headline, summary, highlights, and location.

How it is built: A stateless data-render component that imports portfolioData.profile and shared Win98 icons. It has no custom CSS file and relies on shared layout classes.

OS integration: No useOs and no payload. It is launched from the Portfolio Start menu group and reads only src/data/portfolioData.ts.

Evidence	Current source
Registry	src/data/apps.ts appDefinitions.about; defaultRect x:140,y:80,width:460,height:360; singleton true; safeModeAvailable true
Component	src/components/apps/AboutApp.tsx
Styles	shared global classes
Markdown source	docs/apps/about.md

9.2 Contact (contact)

The "get in touch" card - email, availability, and external links, with a ready-made mailto: button.

What users can do: The contact card: email, availability, summary, location, mailto action, and external links.

How it is built: A stateless component that maps portfolioData.contact.links to anchors and builds a mailto href from the stored email.

OS integration: No useOs and no payload. It is safe-mode available because it is static portfolio content.

Evidence	Current source
Registry	src/data/apps.ts appDefinitions.contact; defaultRect x:310,y:150,width:430,height:330; singleton true; safeModeAvailable true
Component	src/components/apps/ContactApp.tsx
Styles	src/components/apps/ContactApp.css
Markdown source	docs/apps/contact.md

9.3 My Projects (projects)

The project browser - a file-list of every portfolio project; click one to open its detail window. Demonstrates the parent->child app-open pattern.

What users can do: The project browser: Explorer-style rows for portfolioData.projects plus buttons to open details or credits.

How it is built: Unlike most apps, it receives a narrow openApp prop from renderAppWindow rather than calling useOs. Rows are generated directly from portfolioData.projects.

OS integration: Calls openApp('projectDetails', { projectId }) for the selected project and openApp('credits') for attributions. Project Details is non-singleton so multiple project windows can coexist.

Evidence	Current source
Registry	src/data/apps.ts appDefinitions.projects; defaultRect x:190,y:94,width:580,height:430; singleton true; safeModeAvailable true
Component	src/components/apps/ProjectsApp.tsx
Styles	src/components/apps/ProjectsApp.css
Markdown source	docs/apps/projects.md

9.4 Project Details (projectDetails)

The single-project view opened from My Projects - summary, write-up, tech-stack tags, and demo/source links for one project.

What users can do: The detail view for a single portfolio project: name, summary, write-up, tech stack tags, and demo/source links.

How it is built: Receives projectId as a custom prop, looks up the matching portfolioData.projects entry, and falls back to the first project if the payload is missing or unknown.

OS integration: Opened by Projects through a payload. Store title logic keys the window by project id and names it after the project.

Evidence	Current source
Registry	src/data/apps.ts appDefinitions.projectDetails; defaultRect x:360,y:116,width:480,height:360; singleton false; safeModeAvailable true
Component	src/components/apps/ProjectDetailsApp.tsx
Styles	src/components/apps/ProjectDetailsApp.css
Markdown source	docs/apps/project-details.md

9.5 Credits (credits)

The attributions screen - every library, tool, and asset the project leans on, grouped by section with links back to the source.

What users can do: The attribution screen for libraries, tooling, assets, and game/emulation sources.

How it is built: A stateless grouped-list component. It derives section names with a Set, filters credits per section, and renders external links with notes.

OS integration: No useOs and no payload. It is opened from Projects and from the Start menu.

Evidence	Current source
Registry	src/data/apps.ts appDefinitions.credits; defaultRect x:280,y:110,width:470,height:320; singleton true; safeModeAvailable true
Component	src/components/apps/CreditsApp.tsx
Styles	src/components/apps/CreditsApp.css
Markdown source	docs/apps/credits.md

9.6 Help (help)

The desktop's built-in WinHelp-style guide - a three-topic browser covering how to use the OS, what each program does, and every MS-DOS Prompt command.

What users can do: A WinHelp-style built-in guide with Getting Started, Programs, and MS-DOS Prompt Commands topics.

How it is built: A single selected-topic state switches between static articles. TOPICS and DOS_COMMANDS are module-level arrays rendered into nav buttons and command tables.

OS integration: No live OS actions. It imports portfolioData.profile.name for the welcome heading and status bar and is available in Safe Mode.

Evidence	Current source
Registry	src/data/apps.ts appDefinitions.help; defaultRect x:200,y:76,width:640,height:470; singleton true; safeModeAvailable true
Component	src/components/apps/HelpApp.tsx
Styles	src/components/apps/HelpApp.css
Markdown source	docs/apps/help.md

10 Cross-App Patterns Worth Noting

Pattern	Where it appears	Why it matters
Shared save contract	Notepad, WordPad, Paint, Sound Recorder	All writes return error string or null through fsOps, so UI errors are handled consistently.
Payload-driven windows	Explorer path, editor filePath, IE url, Terminal command, Project Details projectId	One WindowPayload channel supports many app launch scenarios without coupling app components together.
Read-only derived views	Gallery, Image Viewer, Media Player, Video Player, Credits, Help	Views reflect the current data source without maintaining duplicate app state.
Pure local apps	Calculator, Minesweeper, About, Contact, Credits, Help	Small windows stay independent of the OS facade when they do not need it.
Imperative browser APIs isolated behind refs	Paint, Sound Recorder, Task Manager graphs, Media/Video players	High-frequency canvas/media work avoids noisy React state loops.
Safe Mode gating	IE, Media Player, Video Player, Sound Recorder, Network, DOS Game	Apps depending on network, emulator assets, or richer media are blocked in Safe Mode by registry metadata.

11 Source Evidence Used

Source set	How it was used
docs/apps/*.md	Read as app-level intent, feature descriptions, and existing implementation notes.
src/data/apps.ts	Verified the exact app registry, default rectangles, singleton flags, Safe Mode flags, desktop icons, and Start menu entries.
src/App.tsx	Verified lazy imports, renderAppWindow cases, keyed remount behavior, and phase rendering.
src/components/apps/*App.tsx	Verified the implementation model, hooks, useOs calls, browser APIs, and app-to-app handoffs for every app.
src/os/filesystem.ts	Verified file extension associations and shared VFS behavior used by apps.
src/os/commands.ts	Verified terminal command labels, effects, streams, and app-launch aliases.
src/data/portfolioData.ts	Verified About, Contact, Projects, Project Details, Credits, and Help content sources.

Design preset: compact_reference_guide. Page geometry: US Letter, 1 inch margins, Calibri 11 pt body, fixed-width DXA tables with repeating header rows.